LP ANSWERS CHAPTER-1

1. **Describe with examples, the different ways of assigning values to structure members.**

   Answer:

   There are multiple ways to assign values to structure members in C:

   (a) Direct assignment

```
struct Student {
    int id;
    float marks;
};

struct Student s1;
s1.id = 101;
s1.marks = 85.5;
```

   (b) Initialization at declaration

```
struct Student s1 = {101, 85.5};
```

   (c) Designated initialization (C99)

```
struct Student s1 = {.marks = 85.5, .id = 101};
```

   (d) Using pointer to structure

```
struct Student s1;
struct Student *ptr = &s1;

ptr->id = 101;
ptr->marks = 85.5;
```

2. **Describe the different approaches that can be used to pass structures as function arguments.**

Answer:

(a) Passing entire structure (call by value)

```
void display(struct Student s) {
    printf("%d %.2f", s.id, s.marks);
}
```

- Copy of structure is passed
- Changes do not affect original

(b) Passing structure using pointer (call by reference)

```
void display(struct Student *s) {
    printf("%d %.2f", s->id, s->marks);
}
```

- Efficient (no copying)
- Changes affect original

(c) Passing individual members

```
void display(int id, float marks) {
    printf("%d %.2f", id, marks);
}
```

3. **Create a structure that can describe a restaurant. It should have members that include the name, address, average cost, and type of food. Write a function that displays all restaurants of a given food type in order of cost, with the least costly first.**

Answer:

```
#include <stdio.h>
#include <string.h>

struct Restaurant {
    char name[50];
    char address[100];
    float cost;
    char type[30];
};

void  displayByType(struct  Restaurant  r[],  int  n,  char
foodType[]) {
    struct Restaurant temp;

    // Sort by cost (ascending)
    for(int i = 0; i < n-1; i++) {
        for(int j = i+1; j < n; j++) {
            if(r[i].cost > r[j].cost) {
                temp = r[i];
                r[i] = r[j];
                r[j] = temp;
            }
        }
    }

    printf("\nRestaurants serving %s:\n", foodType);

    for(int i = 0; i < n; i++) {
        if(strcmp(r[i].type, foodType) == 0) {
            printf("%s   |   %s   |   %.2f\n",   r[i].name,
r[i].address, r[i].cost);
        }
    }
}
```

```
int main() {
    struct Restaurant r[3] = {
        {"Spice Hub", "Mumbai", 250, "Indian"},
        {"Pizza Point", "Delhi", 300, "Italian"},
        {"Food Corner", "Pune", 150, "Indian"}
    };

    displayByType(r, 3, "Indian");
    return 0;
}
```

4. Explain Dynamic memory allocation with an example

Answer:

Dynamic Memory Allocation (DMA)

Definition:
Dynamic Memory Allocation is the process of allocating memory at **runtime (heap memory)** using functions like malloc(), calloc(), realloc(), and free().

It allows flexible memory usage when size is not known in advance.
Memory is allocated during program execution and must be released manually.

1. malloc()

`malloc()` allocates a single block of memory of given size.
It does **not initialize** the memory (contains garbage values).

Syntax & Initialization:

```
ptr = (type*) malloc(n * sizeof(type));
```

Example:

```
int *ptr;
ptr = (int*) malloc(5 * sizeof(int));
```

2. calloc()

`calloc()` allocates memory for multiple elements.
It **initializes all values to 0** by default.

Syntax & Initialization:

```
ptr = (type*) calloc(n, sizeof(type));
```

Example:

```
int *ptr;
ptr = (int*) calloc(5, sizeof(int));
```

3. realloc()

`realloc()` resizes previously allocated memory.
It can increase or decrease the memory size while keeping existing data.

Syntax & Initialization:

```
ptr = (type*) realloc(ptr, new_size);
```

Example:

```
ptr = (int*) realloc(ptr, 10 * sizeof(int));
```

4. free()

`free()` releases dynamically allocated memory back to the system.
It helps avoid memory leaks.

Syntax:

```
free(ptr);
```

Example:

```
free(ptr);
```

5. For the following set of sample data, write a modular C program using pointers to compute the standard deviation and the mean.-6, -12, 8, 13, 11, 6, 7, 2, -6, -9, -10, 11, 10, 9, 2

Answer:

```
#include <stdio.h>

#include <math.h>

#define N 15

// Function to calculate mean

float calculateMean(int *arr, int n) {

    int i;

    float sum = 0;

    for(i = 0; i < n; i++) {

        sum += *(arr + i);   // using pointer

    }

    return sum / n;

}

// Function to calculate standard deviation

float calculateStdDev(int *arr, int n, float mean) {

    int i;

    float sum = 0;

    for(i = 0; i < n; i++) {

        sum += pow((*(arr + i) - mean), 2);

    }
```

```
    return sqrt(sum / n);
}
int main() {
    int data[N] = {-6, -12, 8, 13, 11, 6, 7, 2, -6, -9, -
10, 11, 10, 9, 2};
    float mean, stddev;
    mean = calculateMean(data, N);
    stddev = calculateStdDev(data, N, mean);
    printf("Mean = %.2f\n", mean);
    printf("Standard Deviation = %.2f\n", stddev);
    return 0;
}
```

6. Marks obtained by a batch of students in the Examination are tabulated as follows: Student name Marks obtained Laxmi 70 80 83 87 Vivek 77 89 90 96 . Write a modular C program using pointer to compute the total marks obtained by each student and print the rank list. give question in correct order and c program.

Answer:

```
#include <stdio.h>
#include <string.h>
#define SUBJECTS 4
#define MAX 10

// Structure for student
struct Student {
```

```
    char name[20];
    int marks[SUBJECTS];
    int total;
};
// Function to calculate total marks
void calculateTotal(struct Student *s, int n) {
    int i, j;
    for(i = 0; i < n; i++) {
        (s + i)->total = 0;
        for(j = 0; j < SUBJECTS; j++) {
            (s + i)->total += (s + i)->marks[j];
        }
    }
}
// Function to sort students by total marks (descending)
void sortStudents(struct Student *s, int n) {
    int i, j;
    struct Student temp;
    for(i = 0; i < n - 1; i++) {
        for(j = i + 1; j < n; j++) {
            if((s + i)->total < (s + j)->total) {
                temp = *(s + i);
                *(s + i) = *(s + j);
                *(s + j) = temp;
```

```
            }
        }
    }
}
// Function to display rank list
void display(struct Student *s, int n) {
    int i;
    printf("\nRank List:\n");
    for(i = 0; i < n; i++) {
        printf("Rank %d: %s - Total = %d\n", i + 1, (s +
i)->name, (s + i)->total);
    }
}
int main() {
    struct Student s[MAX];
    int n, i, j;
    printf("Enter number of students: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("\nEnter name of student %d: ", i + 1);
        scanf("%s", (s + i)->name);
        printf("Enter marks in %d subjects: ", SUBJECTS);
        for(j = 0; j < SUBJECTS; j++) {
            scanf("%d", &(s + i)->marks[j]);
```

```
            }
        }

        calculateTotal(s, n);
        sortStudents(s, n);
        display(s, n);
        return 0;
    }
```

7. **Question 7 Home Work**

8. **The X and Y coordinates of N different points are entered through the keyboard. Write a modular C program using pointers to find the distance between the first point and the last point.**

Answer:

First point → `p[0]`

Last point → `p[n-1]`

```
#include <stdio.h>
#include <math.h>
struct Point {
    float x, y;
};
```

```
// function to find distance
float distance(struct Point *p, int n) {
    float dx = (p + n - 1)->x - p->x;
    float dy = (p + n - 1)->y - p->y;
    return sqrt(dx*dx + dy*dy);
}
int main() {
    int n, i;
    struct Point p[20];
    printf("Enter number of points: ");
    scanf("%d", &n);
    for(i = 0; i < n; i++) {
        printf("Enter x and y: ");
        scanf("%f %f", &(p+i)->x, &(p+i)->y);
    }
    printf("Distance = %.2f", distance(p, n));
    return 0;
}
```

9. Create two structures named Metric and British which store the values of distances. The Metric structurestores the values in meters and centimeters and the British structure stores the values in feet and inches. Write amodular C program that reads values for the structure variables and adds values contained in one variable of Metric to the contents of another variable of British. The program should display the result in the format of feet and inches or meters and centimeters as required.

Answer:

```
#include <stdio.h>

// Structure definitions

struct Metric {

    int meter;

    int cm;

};

struct British {

    int feet;

    int inch;

};

// Function to convert Metric to British

struct British convertToBritish(struct Metric m) {

    struct British b;

    float totalInches = (m.meter * 100 + m.cm) / 2.54;

    b.feet = (int)(totalInches / 12);

    b.inch = (int)totalInches % 12;


    return b;

}


// Function to add two British distances

struct British add(struct British b1, struct British b2)
{
```

```
    struct British result;
    result.feet = b1.feet + b2.feet;
    result.inch = b1.inch + b2.inch;
    // Convert inches to feet if >12
    if(result.inch >= 12) {
        result.feet += result.inch / 12;
        result.inch = result.inch % 12;
    }
    return result;
}
// Display function
void display(struct British b) {
    printf("Result = %d feet %d inches\n", b.feet,
b.inch);
}
int main() {
    struct Metric m;
    struct British b1, b2, result;
    // Input Metric
    printf("Enter meters and centimeters: ");
    scanf("%d %d", &m.meter, &m.cm);



    // Input British
```

```
    printf("Enter feet and inches: ");
    scanf("%d %d", &b1.feet, &b1.inch);
    // Convert Metric to British
    b2 = convertToBritish(m);
    // Add
    result = add(b1, b2);
    // Display
    display(result);
    return 0;
}
```

10. Question 10 Home Work